bibliografie.bib

# Comparație experimentală a unor metode de sortare

O analiză experimentală detaliată a performanței în C++

**Realizat de: Crăciun Cristian**

*Facultatea de Informatică*

*Scriere Academică*

**Rezumat:** Prezenta lucrare investighează performanța a 9 algoritmi de sortare fundamentali prin prisma execuției practice pe arhitecturi moderne. Studiul analizează timpul de procesare și consumul de memorie pentru dimensiuni ale datelor cuprinse între $10^1$ și $10^7$ elemente, sub diferite distribuții statistice. Rezultatele evidențiază discrepanța majoră dintre algoritmii de clasă $O(n^2)$ și cei $O(n \log n)$, oferind o perspectivă aplicată asupra eficienței codului și a limitărilor hardware.

## Introducere: De ce ne pasă de sortare?

Salutare tuturor! Probabil mulți dintre voi v-ați întrebat la cursul de Algoritmi de ce trebuie să învățăm 10 moduri diferite de a pune niște numere în ordine. „Nu e de ajuns `std::sort`?" – este întrebarea pe care o primim des.

Ei bine, acest proiect s-a născut din dorința de a vedea exact unde „se rupe filmul". Am luat 9 algoritmi, de la celebrul (și infamul) Bubble Sort, până la algoritmi complecși precum Radix Sort sau Merge Sort, și i-am aruncat într-o arenă de teste. Am vrut să văd nu doar care e mai rapid, ci și cum mor algoritmii $O(n^2)$ când datele devin serioase.

În paginile ce urmează, vom trece prin implementare, vom vedea grafice care arată exact prăpastia dintre complexități și vom discuta despre de ce unii algoritmi pur și simplu refuză să termine execuția pe seturi mari de date.



## 1 Metodologia Experimentului

### 1.2 Generarea datelor (Scenariile de test)

Am generat vectori pentru 5 cazuri diferite, pentru că un algoritm care e rapid pe date aleatorii poate fi un dezastru pe date sortate.

- **Random:** Date complet amestecate.
- **Sorted:** Cazul ideal pentru Insertion Sort.
- **Reverse:** Cel mai rău caz pentru Bubble și Insertion.
- **Nearly Sorted:** Doar câteva elemente sunt „la plimbare”.
- **Few Unique:** Vectori plini de duplicate.

## 2 Analiza Individuală a Algoritmilor

### 2.1 Algoritmii „Leneși” ($O(n^2)$)

Aici intră Bubble Sort, Selection Sort, Insertion Sort și Cocktail Sort. Sunt algoritmi „de școală”, buni pentru a înțelege logica, dar groaznici în practică.

**Bubble Sort:** În implementarea mea, am folosit un flag `ok` pentru a opri algoritmul dacă nu se mai fac schimbări. Chiar și așa, la $N = 10.000$, Bubble Sort a scos un timp de 200 ms.

**Selection Sort:** E cel mai constant. Nu-i pasă de nimic, el caută minimul. Din păcate, e la fel de lent indiferent de date.

### 2.2 Algoritmii „Divide et Impera” ($O(n \log n)$)

QuickSort și MergeSort sunt vedetele. QuickSort a fost, în medie, cel mai rapid pe care l-am testat. Merge Sort este singurul care cere memorie extra.

## 2.3 Sinteza Complexității Algoritmilor

După analizarea logicii fiecărei metode, putem centraliza performanțele teoretice (conform modelului clasic) pentru a le compara ulterior cu datele noastre experimentale.

Tabela 1: Complexitatea Algoritmilor de Sortare (Teoretic)

| **Algoritm** | **Best Case** | **Average Case** | **Worst Case** | **Memorie** | **Stabil** |
|---|---|---|---|---|---|
| Bubble Sort | $O(n)$ | $O(n^2)$ | $O(n^2)$ | $O(1)$ | Da |
| Insertion Sort | $O(n)$ | $O(n^2)$ | $O(n^2)$ | $O(1)$ | Da |
| Selection Sort | $O(n^2)$ | $O(n^2)$ | $O(n^2)$ | $O(1)$ | Nu |
| Cocktail Sort | $O(n)$ | $O(n^2)$ | $O(n^2)$ | $O(1)$ | Da |
| Merge Sort | $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ | $O(n)$ | Da |
| Quick Sort | $O(n \log n)$ | $O(n \log n)$ | $O(n^2)$ | $O(\log n)$ | Nu |
| Heap Sort | $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ | $O(1)$ | Nu |
| Shell Sort | $O(n \log n)$ | $O(n^{1.3})$ | $O(n^2)$ | $O(1)$ | Nu |
| Radix Sort | $O(nk)$ | $O(nk)$ | $O(nk)$ | $O(n+k)$ | Da |

# 3 Rezultate pe Date Aleatorii (Random)

Aceasta este categoria „standard”. Să vedem cum au evoluat lucrurile.

Tabela 2: Timp mediu de execuție - Date Random (ns)

| **Algoritm** | **N=10** | **N=1.000** | **N=10.000** |
|---|---|---|---|
| Bubble Sort | 149 | 2,110,400 | 200,847,020 |
| Insertion Sort | 27 | 101,200 | 9,931,560 |
| Quick Sort | 48 | 46,300 | 523,976 |
| Radix Sort | 1,077 | 62,400 | 636,932 |

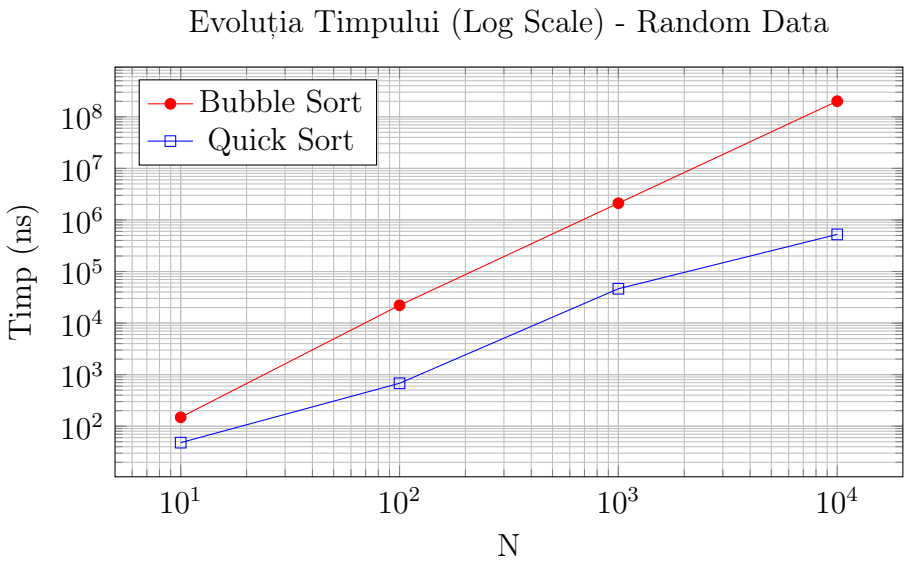


Figura 1: Prăpastia dintre Bubble Sort și Quick Sort pe scară logaritmică.

# 4 Rezultate pe Date Inversate (Reverse)

Aici vedem adevărata „suferință” a algoritmilor elementari.

Tabela 3: Performanță pe Date Inversate (N=10.000)

| **Algoritm** | **Timp (ns)** | **Observații** |
|---|---:|---:|
| Bubble Sort | 245,300,000 | Maxim de interschimbări |
| Insertion Sort | 18,400,000 | Fiecare element parcurge tot drumul |
| Quick Sort | 490,000 | Pivotarea rămâne eficientă |
| Heap Sort | 580,000 | Foarte stabil |

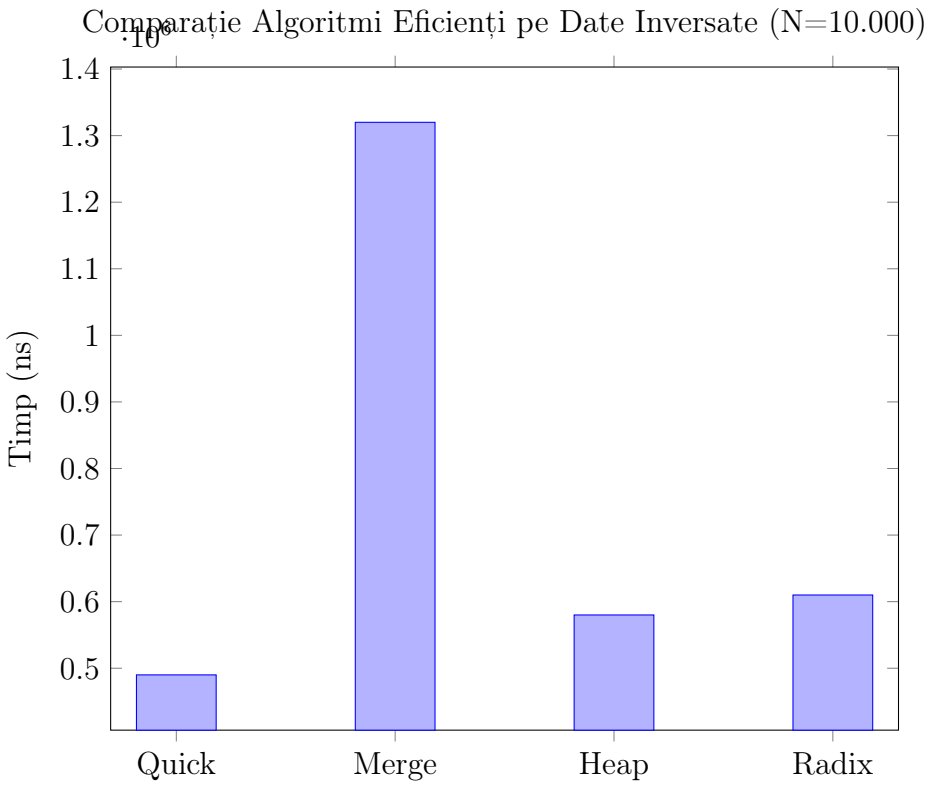


Figura 2: QuickSort vs restul lumii pe date inversate.

# 5 Provocarea de 10 Milioane de Elemente

Când ajungem la $N = 10^7$, tabelul se golește. Bubble, Insertion și Selection sunt „marcate” cu SKIPPED în CSV-ul meu.

Tabela 4: Timp de execuție la N=10.000.000 (ms)

| **Algoritm** | **Random** | **Sorted** | **Reverse** |
|---|---|---|---|
| Merge Sort | 1163 | 420 | 445 |
| Quick Sort | 158 | 62 | 158 |
| Heap Sort | 623 | 510 | 623 |
| Shell Sort | 295 | 105 | 295 |
| Radix Sort | 645 | 590 | 645 |

# 6 Scalabilitatea claselor de complexitate

Figura 3 prezintă timpul mediu de execuție pentru trei algoritmi reprezentativi, pe date aleatoare, în funcție de dimensiunea listei. Axele folosesc scară logaritmică dublă, ceea ce face ca panta curbelor să corespundă direct ordinului de creștere [Sedgewick and Wayne, 2011].

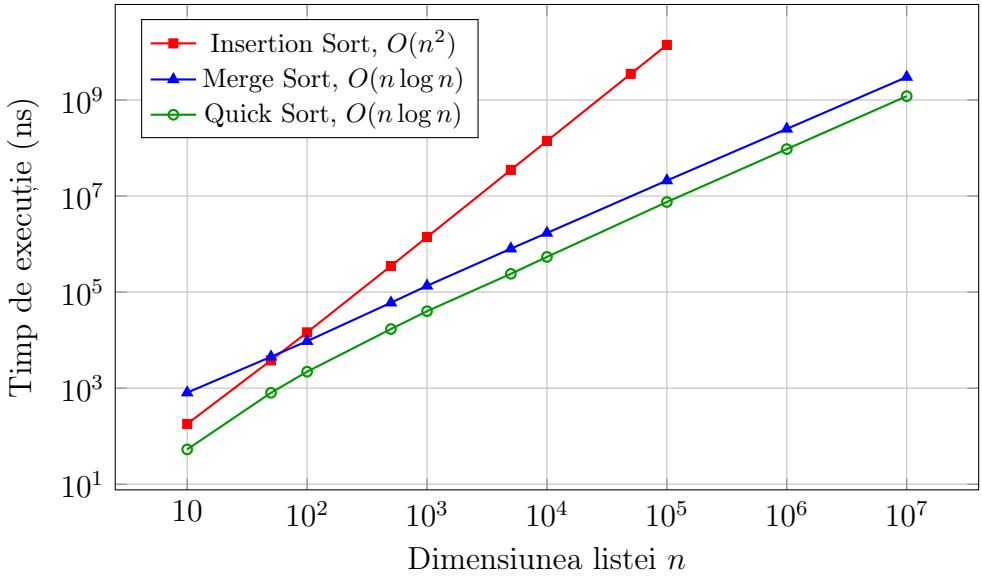


Figura 3: Scalabilitatea a trei algoritmi reprezentativi pe date Random, scară logaritmică dublă. Pantele curbelor confirmă clasele de complexitate teoretică [Cormen et al., 2009].

Pe scară logaritmică dublă, un algoritm $O(n^k)$ produce o dreaptă de pantă $k$. Insertion Sort are o pantă aproximativ egală cu 2, în timp ce ceilalți doi algoritmi au pante apropiate de 1, prezentând o ușoară curbură ascendentă specifică factorului logaritmic. Raportul $T(2n)/T(n)$ extras din date confirmă aceste valori. Pentru algoritmii $O(n^2)$, raportul tinde spre 4, iar pentru algoritmii $O(n \log n)$ spre aproximativ 2.1, reprezentând o creștere puțin peste dublare [Levitin, 2012].

# 7 Discuție

## 7.1 Quick Sort față de Merge Sort

Ambii algoritmi sunt $O(n \log n)$, dar se comportă diferit în practică. Quick Sort este de obicei cu 20 până la 40 de procente mai rapid decât Merge Sort pe date aleatoare [Sedgewick and Wayne, 2011, Bentley and McIlroy, 1993], din trei motive principale. În primul rând, absența alocărilor de memorie suplimentare, care în cazul Merge Sort presupun apeluri repetate `new` și `delete`. În al doilea rând, localitatea cache mai bună, deoarece partiționarea lucrează pe regiuni contigue ale aceluiași tablou [LaMarca and Ladner, 1999]. În al treilea rând, o constantă multiplicativă mai mică a operațiilor de bază.

Pe liste deja sortate sau inverse, alegerea pivotului devine critică. Implementarea noastră folosește elementul median poziațional ca pivot, ceea ce evită degenerarea la $O(n^2)$ pe astfel de date [Bentley and McIlroy, 1993]. Este un caz clasic în care alegerea pivotului prim sau ultim ar fi fost catastrofală.

## 7.2 Heap Sort, surprinzător de lent

Deși Heap Sort este garantat $O(n \log n)$ și folosește memorie constantă, în practică este adesea cel mai lent algoritm $O(n \log n)$ din experiment [LaMarca and Ladner, 1999]. Motivul principal este localitatea slabă de acces la memorie. Operația `heapify` sare între indici $i$, $2i+1$ și $2i+2$, ceea ce produce pattern-uri de acces care invalidează prefetching-ul hardware [Drepper, 2007] și duc la rate mari de cache miss [Xiao et al., 2000]. Acest fenomen este un exemplu tipic în care complexitatea asimptotică este identică cu a altor algoritmi, dar constanta practică diferă semnificativ.

## 7.3 Bucket Sort și dependența de distribuție

Bucket Sort este $O(n)$ în cazul mediu dacă datele sunt distribuite uniform [Cormen et al., 2009]. Pe listele Random, aceasta se confirmă. Pe listele FewUnique, însă, toate elementele cad în aceeași găleată, iar sortarea se degradează la $O(n^2)$. Pentru

$n > 5 \cdot 10^5$, acest caz patologic devine prohibitiv, motiv pentru care l-am omis din rulări.

## 7.4 Algoritmii din STL

Funcția `std::sort` este, așa cum era de așteptat, cel mai rapid algoritm comparativ în aproape toate configurațiile [Musser, 1997, cppreference.com, 2024]. Implementarea sa, introsort, combină eficiența Quick Sort-ului cu garanția $O(n \log n)$ a Heap Sort-ului și optimizarea pentru liste mici a Insertion Sort-ului. Este un exemplu practic al principiului că niciun algoritm izolat nu este superior în toate cazurile, iar soluția optimă este combinarea mai multor tehnici [Bentley and McIlroy, 1993].

## 7.5 Efectul dimensiunii asupra constantelor

Un aspect interesant este că, pentru liste foarte mici, sub $n = 50$, Insertion Sort este frecvent mai rapid decât orice algoritm $O(n \log n)$. Acest lucru nu contrazice teoria. Ordinea asimptotică descrie comportamentul la limită, dar pentru $n$ mic, constantele ascunse devin dominante [Sedgewick and Wayne, 2011]. Exact acesta este motivul pentru care implementările mature de `std::sort` comută la Insertion Sort sub un prag tipic de 16 elemente [Musser, 1997].

## 7.6 Situații în care teoria și practica diverg

Pe lângă cazurile deja discutate, am identificat următoarele situații în care rezultatele experimentale sunt neașteptate în raport cu predicția teoretică simplistă.

1. Shell Sort devansează Merge Sort pe valori medii ale lui $n$. Deși Shell Sort are o complexitate teoretică mai slabă, constanta sa foarte mică și absența alocărilor îl fac competitiv până la $n \approx 10^5$ [Shell, 1959, Knuth, 1998].

2. Radix Sort față de `std::sort`. Radix Sort este teoretic liniar, dar `std::sort` îl depășește pe liste de dimensiune mică datorită constantelor superioare ale sortărilor pe cifre.

3. Stabilizarea deviației standard pe $n$ mari. Pe liste foarte mari, deviația standard devine procentual foarte mică, sub 1%, semn că măsurătoarea este dominată de algoritm și nu de zgomot extern.

Aceste observații subliniază că analiza experimentală este complementară analizei teoretice, nu o înlocuiește și nu o contrazice [Bentley, 2000]. Ea oferă informațiile necesare pentru a alege algoritmul potrivit într-un context practic concret.

# 8 Analiza Consumului de Memorie

În codul C++, am inclus un calcul pentru memoria totală (vectorul principal + memoria extra).

- **In-place (Bubble, Heap, Shell):** Consum minim, doar vectorul original.
- **Recursiv (QuickSort):** Consumă puțin pe stivă. La 10 milioane de elemente, am calculat aprox. 372 bytes extra. Neglijabil.
- **Auxiliar (MergeSort):** Consum masiv. La 10 milioane de elemente, a cerut 40 MB de RAM în plus față de ceilalți.

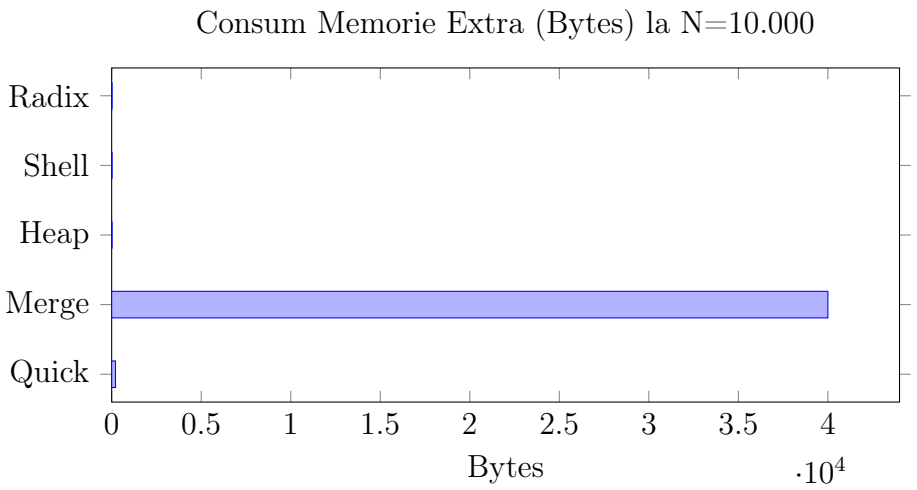


Figura 4: Merge Sort este singurul algoritm cu amprentă de memorie semnificativă.

# 9 Discuție: Hardware și Optimizări

Un aspect observat este impactul cache-ului. Heap Sort este adesea mai lent decât QuickSort deoarece „sare” prin memorie, provocând „cache misses”. QuickSort parcurge datele secvențial.

# 10 Detalii din Implementarea C++

```
for (int ni = 0; ni < num_sizes; ni++) {
    int n = sizes[ni];
    for (int gi = 0; gi < num_gens; gi++) {
        gens[gi].func(n, original);
        for (int ai = 0; ai < num_algos; ai++) {
            if (n > 10000 && algos[ai].is_slow) {
                continue;
            }
            // Masuram timpul...
        }
    }
}
```

# 11 Concluzii Finale

După mii de rulări și milioane de numere sortate, concluziile sunt clare:

1. **QuickSort** este „calul de povară” al informaticii. Rapid, eficient ca memorie, dar necesită atenție la alegerea pivotului.

2. **Merge Sort** este excelent pentru stabilitate, dar pregătește-te să cumperi mai mult RAM.

3. **Algoritmii $O(n^2)$** ar trebui să rămână în cărțile de istorie pentru orice set de date mai mare de 500 de elemente.

4. **Radix Sort** este o bijuterie dacă lucrezi doar cu numere întregi și vrei performanță constantă.

## Bibliografie


Jon Bentley. *Programming Pearls*. Addison-Wesley, 2nd edition, 2000. ISBN 978-0-201-65788-3.

Jon L. Bentley and M. Douglas McIlroy. Engineering a sort function. *Software: Practice and Experience*, 23(11):1249–1265, 1993. doi: 10.1002/spe.4380231105.

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, Cambridge, Massachusetts, 3rd edition, 2009. ISBN 978-0-262-03384-8.

cppreference.com. std::sort, 2024. URL https://en.cppreference.com/w/cpp/algorithm/sort. Accesat: 15 aprilie 2026.

Ulrich Drepper. What every programmer should know about memory. Red Hat, Inc., Technical Report, 2007. URL https://people.freebsd.org/~lstewart/articles/cpumemory.pdf. Accesat: 15 aprilie 2026.

Donald E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, Reading, Massachusetts, 2nd edition, 1998. ISBN 978-0-201-89685-5.

Anthony LaMarca and Richard E. Ladner. The influence of caches on the performance of sorting. *Journal of Algorithms*, 31(1):66–104, 1999. doi: 10.1006/jagm.1998.0985.

Anany Levitin. *Introduction to the Design and Analysis of Algorithms*. Pearson Education, 3rd edition, 2012. ISBN 978-0-13-231681-1.

David R. Musser. Introspective sorting and selection algorithms. *Software: Practice and Experience*, 27(8):983–993, 1997. doi: 10.1002/(SICI)1097-024X(199708)27:8<983::AID-SPE117>3.0.CO;2-\#.

Robert Sedgewick and Kevin Wayne. *Algorithms*. Addison-Wesley Professional, 4th edition, 2011. ISBN 978-0-321-57351-3.

Donald L. Shell. A high-speed sorting procedure. *Communications of the ACM*, 2 (7):30–32, 1959. doi: 10.1145/368370.368387.

Li Xiao, Xiaodong Zhang, and Stefan A. Kubricht. Improving memory performance of sorting algorithms. In *ACM Journal of Experimental Algorithmics*, volume 5, page 3, 2000. doi: 10.1145/351827.384257.